

**MINISTERUL EDUCAȚIEI ȘI CERCETĂRII  
UNIVERSITATEA „OVIDIUS” DIN CONSTANȚA  
FACULTATEA DE MATEMATICĂ ȘI INFORMATICĂ  
SPECIALIZAREA INFORMATICĂ**

**LUCRARE DE LICENȚĂ**

**Coordonator științific:**

[Titlu, Nume coordonator]

**Absolvent:**

[Numele studentului]

**CONSTANȚA**

**2026**

**MINISTERUL EDUCAȚIEI ȘI CERCETĂRII  
UNIVERSITATEA „OVIDIUS” DIN CONSTANȚA  
FACULTATEA DE MATEMATICĂ ȘI INFORMATICĂ  
SPECIALIZAREA INFORMATICĂ**

## **LUCRARE DE LICENȚĂ**

**Dezvoltarea unei aplicații web pentru gestionarea unui cabinet  
veterinar**

**Studiu de caz: platforma VetCare**

**Coordonator științific:**

[Titlu, Nume coordonator]

**Absolvent:**

[Numele studentului]

**CONSTANȚA  
2026**

## Rezumat

Lucrarea prezintă proiectarea și implementarea unei aplicații web full-stack pentru gestionarea unui cabinet veterinar, denumită VetCare. Aplicația oferă funcții publice de prezentare a cabinetului, formular de contact, autentificare, programare online, profil utilizator și panou administrativ. Tema este motivată de nevoia digitalizării serviciilor locale de îngrijire veterinară, în care comunicarea rapidă, evidența programărilor și accesul controlat la date au impact direct asupra calității relației dintre cabinet și clienți.

Soluția este dezvoltată cu Angular 19 pe partea de frontend, Node.js și Express pe partea de backend și MySQL pentru persistența datelor. Sistemul folosește JSON Web Tokens pentru autentificare, bcrypt pentru protejarea parolelor, validări server-side, rate limiting pentru endpointurile sensibile și constrângeri de bază de date pentru păstrarea integrității datelor. Contribuția proprie constă în structurarea funcțională a aplicației, implementarea fluxului de programare cu verificarea sloturilor ocupate, separarea rolurilor utilizator/admin și realizarea unei documentații tehnice coerente.

Cuvinte cheie: aplicație web, cabinet veterinar, Angular, Node.js, Express, MySQL, JWT, securitate web, programări online.

## Abstract

This thesis presents the design and implementation of a full-stack web application for managing a veterinary clinic, named VetCare. The application provides public presentation pages, a contact form, authentication, online appointment booking, user profile management and an administrative dashboard. The topic is motivated by the need to digitize local veterinary services, where fast communication, appointment traceability and controlled access to data improve the relationship between the clinic and its clients.

The solution is implemented using Angular 19 for the frontend, Node.js and Express for the backend, and MySQL for data persistence. The system uses JSON Web Tokens for authentication, bcrypt for password protection, server-side validation, rate limiting for sensitive endpoints and relational constraints for data integrity. The original contribution consists of the functional design of the system, the appointment workflow with occupied-slot validation, role-based separation between regular users and administrators, and a coherent technical documentation of the software product.

Keywords: web application, veterinary clinic, Angular, Node.js, Express, MySQL, JWT, web security, online appointments.

## Cuprins

| Secțiune                                      | Pagina |
|-----------------------------------------------|--------|
| 1. Introducere                                | 1      |
| 2. Analiza domeniului și definirea cerințelor | 5      |
| 3. Tehnologii utilizate                       | 11     |
| 4. Arhitectura și proiectarea sistemului      | 17     |
| 5. Proiectarea bazei de date și a API-ului    | 24     |
| 6. Implementarea aplicației VetCare           | 31     |
| 7. Testare, validare și evaluare              | 38     |
| 8. Concluzii și direcții viitoare             | 45     |
| Bibliografie                                  | 48     |
| Anexe                                         | 50     |

Notă: numerotarea paginilor este orientativă în această versiune draft. În Word, cuprinsul poate fi actualizat după completarea numelui studentului, a coordonatorului și a eventualelor capturi de ecran finale.

## Lista figurilor

| Nr.      | Descriere                                 |
|----------|-------------------------------------------|
| Figura 1 | Arhitectura generală a aplicației VetCare |
| Figura 2 | Fluxul de autentificare cu JWT            |
| Figura 3 | Modelul relațional al bazei de date       |
| Figura 4 | Fluxul programării online                 |
| Figura 5 | Structura componentelor Angular           |

## Lista tabelelor

| Nr.       | Descriere                 |
|-----------|---------------------------|
| Tabelul 1 | Cerințe funcționale       |
| Tabelul 2 | Cerințe nefuncționale     |
| Tabelul 3 | Tehnologii utilizate      |
| Tabelul 4 | Structura tabelelor MySQL |
| Tabelul 5 | Endpointuri API           |

|           |                  |
|-----------|------------------|
| Tabelul 6 | Scenarii de test |
|-----------|------------------|

# 1. Introducere

Digitalizarea serviciilor medicale și conexe a devenit o cerință firească pentru organizațiile care lucrează direct cu publicul. În cazul cabinetelor veterinare, această transformare este vizibilă prin nevoia de programări online, comunicare rapidă, evidență a pacienților și reducerea timpului petrecut la telefon pentru operații administrative simple. Un cabinet veterinar modern nu mai funcționează doar ca spațiu fizic, ci și ca serviciu digital disponibil pentru clienți în afara programului clasic.

Tema lucrării se încadrează în zona aplicațiilor web full-stack și urmărește realizarea unei platforme pentru gestionarea relației dintre un cabinet veterinar și clienții săi. Aplicația VetCare oferă o interfață publică prin care vizitatorii pot afla informații despre cabinet, pot trimite mesaje și pot crea conturi, dar include și componente protejate pentru programări, profil utilizator și administrare. Din punct de vedere informatic, proiectul combină noțiuni de analiză a cerințelor, proiectare de baze de date, programare frontend, dezvoltare backend, securitate și testare.

Motivația personală pentru alegerea temei este legată de caracterul practic al aplicației. Spre deosebire de un proiect strict demonstrativ, VetCare pornește de la un scenariu clar: un client dorește să programeze un animal la consultație, iar administratorul cabinetului trebuie să urmărească rapid programările, utilizatorii și mesajele primite. Această situație obligă aplicația să trateze probleme reale, cum ar fi autentificarea, evitarea conflictelor de orar, validarea datelor introduse și diferențierea drepturilor de acces.

Lucrarea are ca obiectiv principal proiectarea și implementarea unei aplicații web care să fie ușor de folosit, coerentă la nivel de arhitectură și suficient de sigură pentru un context educațional. Obiectivele secundare sunt definirea cerințelor funcționale și nefuncționale, alegerea unei stive tehnologice potrivite, proiectarea bazei de date, implementarea API-ului REST, realizarea interfeței Angular și descrierea unui plan de testare care să acopere fluxurile importante.

Din punct de vedere istoric, aplicațiile web au evoluat de la pagini statice la sisteme complexe în care clientul și serverul colaborează permanent. Separarea dintre frontend și backend permite dezvoltarea independentă a interfeței și a logicii de business. VetCare folosește această separare printr-un frontend Angular care comunică prin HTTP cu un backend Express, iar backend-ul păstrează datele într-o bază MySQL. Această arhitectură este potrivită pentru aplicații educaționale, dar și pentru produse mici sau medii care pot fi extinse ulterior.

Contribuția proprie constă în definirea fluxului complet al aplicației, implementarea componentelor principale și documentarea deciziilor tehnice. Aplicația nu este prezentată ca un sistem clinic complet, ci ca un nucleu funcțional care poate fi extins cu evidența animalelor, fișe medicale, facturare sau notificări. În forma actuală, accentul este pus pe programări, comunicare și administrare, acestea fiind funcțiile care demonstrează cel mai bine legătura dintre interfață, API și baza de date.

## **1.1 Structura lucrării**

Capitolul al doilea prezintă analiza domeniului și cerințele aplicației. Sunt discutate actorii principali, cazurile de utilizare și constrângerile de securitate și utilizabilitate. Capitolul al treilea descrie tehnologiile folosite și justifică alegerea lor în raport cu obiectivele proiectului.

Capitolul al patrulea descrie arhitectura generală, fluxurile aplicației și organizarea codului. Capitolul al cincilea se concentrează pe baza de date și pe API-ul backend. Capitolul al șaselea prezintă implementarea concretă, iar capitolul al șaptelea discută testarea. Ultimul capitol sintetizează rezultatele și propune direcții viitoare de dezvoltare.

## 2. Analiza domeniului și definirea cerințelor

Un cabinet veterinar gestionează simultan servicii medicale, comunicare cu proprietarii animalelor și activități administrative. Într-un flux tradițional, multe activități sunt realizate telefonic sau prin registre separate, ceea ce poate genera întârzieri, dublări ale informațiilor și dificultăți în urmărirea istoricului. O aplicație web nu elimină rolul personalului, dar poate automatiza acțiunile repetitive și poate centraliza datele necesare pentru decizii rapide.

Analiza domeniului a pornit de la identificarea actorilor: vizitatorul neautentificat, utilizatorul autentificat și administratorul. Vizitatorul poate consulta informații publice și poate trimite un mesaj prin formularul de contact. Utilizatorul autentificat poate face programări și poate vedea istoricul propriu. Administratorul are acces la dashboard, statistici, lista utilizatorilor, programări și mesaje.

O cerință importantă este separarea clară a drepturilor. Un utilizator obișnuit nu trebuie să poată vedea programările altor persoane sau să schimbe roluri. În același timp, administratorul trebuie să poată gestiona datele operaționale ale cabinetului. Această diferențiere este implementată prin tokenuri JWT care includ rolul utilizatorului și prin middleware backend care verifică accesul înainte de executarea operațiilor sensibile.

### 2.1 Cerințe funcționale

| Cod | Cerință            | Descriere                                                                                           |
|-----|--------------------|-----------------------------------------------------------------------------------------------------|
| CF1 | Înregistrare cont  | Aplicația permite crearea unui cont folosind email, parolă și nume.                                 |
| CF2 | Autentificare      | Utilizatorul se poate autentifica și primește un token JWT.                                         |
| CF3 | Profil utilizator  | Utilizatorul poate vizualiza datele proprii și poate modifica numele sau parola.                    |
| CF4 | Programare online  | Utilizatorul alege data, ora, serviciul și tipul animalului.                                        |
| CF5 | Verificare sloturi | Aplicația afișează orele ocupate pentru data selectată.                                             |
| CF6 | Anulare programare | Utilizatorul poate anula programări proprii, iar administratorul poate gestiona toate programările. |
| CF7 | Formular contact   | Vizitatorii pot trimite mesaje fără autentificare.                                                  |
| CF8 | Dashboard admin    | Administratorul vede statistici despre                                                              |

|      |                        |                                                                    |
|------|------------------------|--------------------------------------------------------------------|
|      |                        | utilizatori, programări și mesaje.                                 |
| CF9  | Gestionare utilizatori | Administratorul poate schimba roluri și poate șterge conturi.      |
| CF10 | Gestionare mesaje      | Administratorul poate marca mesajele ca citite și le poate șterge. |

Cerințele funcționale au fost alese astfel încât aplicația să acopere ciclul minim al interacțiunii dintre client și cabinet. Accentul cade pe programare și comunicare, deoarece acestea sunt procesele care se repetă frecvent și care pot fi simplificate printr-o platformă web.

Fluxul programării are o importanță specială. Sistemul trebuie să împiedice alegerea zilelor din trecut, a weekendurilor și a intervalelor în afara programului. De asemenea, același slot nu trebuie să fie acceptat de două ori. Această logică este verificată la nivel de backend, deoarece interfața poate fi ocolită prin cereri HTTP directe.

## 2.2 Cerințe nefuncționale

| Cod  | Categorie       | Descriere                                                                      |
|------|-----------------|--------------------------------------------------------------------------------|
| CNF1 | Securitate      | Parolele sunt hash-uite, iar accesul la rute protejate se face cu JWT.         |
| CNF2 | Utilizabilitate | Interfața este în limba română și folosește mesaje clare de eroare.            |
| CNF3 | Performanță     | API-ul trebuie să răspundă rapid pentru operații curente de listare și creare. |
| CNF4 | Mentenabilitate | Codul este separat în componente, rute, controllere, servicii și middleware.   |
| CNF5 | Integritate     | Baza de date folosește chei primare, chei externe și indecși.                  |
| CNF6 | Portabilitate   | Proiectul poate fi rulat local cu Node.js, npm și MySQL.                       |

Cerințele nefuncționale influențează calitatea soluției chiar dacă nu sunt observate imediat de utilizator. De exemplu, o aplicație poate permite autentificarea, dar dacă parolele sunt stocate în clar, soluția este inacceptabilă din punct de vedere al securității. În VetCare, parolele sunt protejate cu bcrypt, iar endpointurile de autentificare sunt limitate pentru a reduce riscul încercărilor repetate.

Mentenabilitatea a fost urmărită prin organizarea proiectului în două aplicații distincte. Frontend-ul conține componente Angular specializate pe pagini și servicii pentru comunicarea cu API-ul. Backend-ul conține rute, controllere, middleware și un adaptor pentru baza de date. Această împărțire facilitează modificările ulterioare, deoarece o schimbare în interfață nu obligă rescrierea logicii de acces la date.

## 2.3 Cazuri de utilizare

- Crearea contului: utilizatorul completează email, nume și parolă; backend-ul validează datele, verifică unicitatea emailului, hash-uieste parola și returnează tokenul.
- Programarea unei consultații: utilizatorul autentificat deschide pagina de programare, alege o zi lucrătoare, selectează un interval liber, completează serviciul și tipul animalului, apoi trimite cererea.
- Administrarea programărilor: administratorul accesează dashboard-ul, vede programările ordonate, filtrează mental după status și poate confirma, anula sau șterge înregistrări.
- Trimiterea unui mesaj de contact: vizitatorul completează formularul public, iar mesajul este stocat în tabela `contact_messages` pentru a fi consultat ulterior de administrator.

Aceste cazuri de utilizare acoperă atât partea publică, cât și partea protejată a aplicației. Ele sunt suficiente pentru demonstrarea unei aplicații full-stack complete, cu fluxuri de citire, scriere, actualizare și ștergere a datelor.

### 3. Tehnologii utilizate

Alegerea tehnologiilor a urmărit obținerea unui echilibru între claritate, productivitate și relevanță pentru dezvoltarea web modernă. Proiectul folosește TypeScript și Angular pentru interfață, Node.js și Express pentru API, MySQL pentru baza de date și biblioteci consacrate pentru autentificare și securitate.

Angular este potrivit pentru aplicații cu mai multe pagini și componente reutilizabile. În VetCare, fiecare pagină importantă este reprezentată de un component standalone: Home, About, Contact, Login, Programare, Profile și Dashboard. Rutarea este definită într-un fișier central, iar protecția paginilor se face cu guard-uri.

Node.js permite rularea JavaScript pe server și este adecvat pentru API-uri HTTP. Express adaugă un strat simplu pentru definirea rutelor și middleware-urilor. În proiect, Express este folosit pentru rutele de autentificare, programări, dashboard și contact.

MySQL a fost ales deoarece modelul aplicației este relațional: utilizatorii au programări, iar mesajele de contact sunt stocate separat. Tabelele folosesc chei primare UUID, indecși și o relație între users și appointments. Această structură este ușor de explicat, testat și extins.

| Tehnologie         | Rol               | Utilizare în proiect                                          |
|--------------------|-------------------|---------------------------------------------------------------|
| Angular 19         | Frontend          | Componente standalone, rutare, formulare și comunicare HTTP.  |
| TypeScript 5.7     | Limbaaj frontend  | Tipare statică și cod mai ușor de întreținut.                 |
| Node.js            | Runtime backend   | Execuția serverului JavaScript și a API-ului.                 |
| Express            | Framework backend | Rute HTTP, middleware și gestionarea erorilor.                |
| MySQL              | Bază de date      | Persistența utilizatorilor, programărilor și mesajelor.       |
| JWT                | Autentificare     | Transmiterea identității și a rolului între client și server. |
| bcryptjs           | Securitate parole | Hashuirea parolelor înainte de stocare.                       |
| express-rate-limit | Protecție API     | Limitarea încercărilor repetate pe rute sensibile.            |

### 3.1 Definiții ale tehnologiilor și conceptelor

| Termen      | Definiție și rol în proiect                                                                                                                                                                                                                     |
|-------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| HTML        | Limbaj de marcare folosit pentru structura paginilor web. În VetCare apare în template-urile componentelor Angular, de exemplu <code>home.component.html</code> , <code>login.component.html</code> și <code>programare.component.html</code> . |
| CSS         | Limbaj de stilizare folosit pentru aspect vizual, layout, culori, spațiere și responsive design. În proiect, fiecare componentă are fișier CSS separat.                                                                                         |
| TypeScript  | Limbaj bazat pe JavaScript, cu tipare statică. În Angular este folosit pentru logica componentelor, servicii, guard-uri și configurarea rutelor.                                                                                                |
| JWT         | JSON Web Token este un format compact pentru transmiterea informațiilor semnate între client și server. În VetCare conține id-ul, emailul și rolul utilizatorului.                                                                              |
| API         | Application Programming Interface reprezintă interfața prin care două aplicații comunică. În VetCare, API-ul backend oferă rute pentru autentificare, programări, mesaje și dashboard.                                                          |
| REST API    | Stil de proiectare API care folosește resurse, URL-uri și metode HTTP. În VetCare, resursele principale sunt users, appointments și contact messages.                                                                                           |
| Endpoint    | Adresă concretă din API care execută o operație, de exemplu <code>POST /api/auth/login</code> sau <code>GET /api/appointments/mine</code> .                                                                                                     |
| HTTP        | Protocolul prin care browserul și serverul schimbă cereri și răspunsuri. Metodele folosite în proiect sunt GET, POST, PUT și DELETE.                                                                                                            |
| HTTP Client | Componentă software care trimite cereri HTTP. În Angular, <code>HttpClient</code> este folosit pentru comunicarea dintre componente/servicii și backend.                                                                                        |
| Request     | Cererea trimisă de client către server. Poate conține metodă, URL, headere, token JWT și body JSON.                                                                                                                                             |
| Response    | Răspunsul serverului către client. Include cod de status, headere și date JSON sau mesaj de eroare.                                                                                                                                             |
| JSON        | Format text pentru schimb de date. VetCare îl folosește pentru request body și response body între Angular și Express.                                                                                                                          |
| CORS        | Mecanism de securitate al browserului care controlează accesul între origini diferite. Backend-ul permite frontend-ul local prin configurarea CORS.                                                                                             |
| Middleware  | Funcție Express executată înaintea rutelor finale. În VetCare, middleware-ul verifică JWT-ul și rolul de administrator.                                                                                                                         |
| Node.js     | Mediu de execuție JavaScript pe server. Rulează aplicația backend VetCare.                                                                                                                                                                      |
| Express     | Framework web pentru Node.js, folosit la definirea rutelor API, middleware-urilor și handlerelor de eroare.                                                                                                                                     |
| Controller  | Componentă backend care conține logica unei operații, cum ar fi                                                                                                                                                                                 |

|              |                                                                                                                                                  |
|--------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
|              | register, login sau obținerea statisticilor.                                                                                                     |
| Model        | Structură care descrie forma datelor aplicației. În VetCare, <code>user.model.js</code> creează obiectul utilizator cu UUID și câmpuri standard. |
| URL / Route  | Adresă sau rută prin care utilizatorul sau frontend-ul accesează o pagină ori un endpoint API.                                                   |
| Bază de date | Sistem de stocare persistentă. VetCare folosește MySQL cu tabele pentru utilizatori, programări și mesaje.                                       |

## 3.2 Frontend Angular

Frontend-ul este responsabil de experiența utilizatorului. În aplicația VetCare, Angular gestionează navigarea între pagini, afișarea calendarului de programări, formularele de login și register, profilul utilizatorului și dashboard-ul administrativ. O parte importantă a interfeței este feedback-ul imediat: butoane dezactivate în timpul trimerii datelor, mesaje de eroare și evidențierea datei selectate.

Componentele standalone reduc dependența de module globale și permit declararea explicită a importurilor necesare fiecărei pagini. Această abordare face structura proiectului mai ușor de urmărit pentru un proiect de licență, deoarece fiecare component are fișier TypeScript, HTML și CSS propriu.

Serviciul de autentificare gestionează tokenul, utilizatorul curent și operațiile de login, register și logout. Separarea acestei logici într-un serviciu previne duplicarea codului în componente și permite extinderea ulterioară cu refresh token, stocare mai sigură sau integrare cu un provider extern.

## 3.3 Backend Node.js și Express

Backend-ul expune un API REST sub prefixul `/api`. Fiecare grup de funcționalități are propriul fișier de rutare: `auth.routes.js`, `appointments.routes.js`, `dashboard.routes.js` și `contact.routes.js`. Această structură ajută la separarea responsabilităților și la localizarea rapidă a unei funcții.

Express permite atașarea middleware-urilor globale, cum ar fi CORS, parsarea JSON și rate limiting. Înainte de rutele protejate, backend-ul verifică tokenul JWT și atașează informațiile despre utilizator la obiectul request. Pentru rutele de administrator, se verifică suplimentar rolul admin.

Serverul tratează și cazurile de eroare. Dacă o rută nu există, răspunsul este 404 cu mesaj în limba română. Dacă apare o eroare internă, middleware-ul final returnează un mesaj generic, evitând expunerea detaliilor tehnice către client.

### **3.4 Bază de date MySQL**

Baza de date conține trei tabele: users, appointments și contact\_messages. Tabela users păstrează conturile, tabela appointments păstrează programările, iar tabela contact\_messages păstrează mesajele primite prin formularul public. Relația principală este users - appointments, de tip unu-la-mulți.

Folosirea cheilor externe permite păstrarea integrității datelor. Dacă un utilizator este șters, programările sale sunt șterse automat prin ON DELETE CASCADE. Această regulă simplifică administrarea și previne apariția programărilor fără proprietar.

## 4. Arhitectura și proiectarea sistemului

Arhitectura aplicației VetCare este de tip client-server. Clientul este aplicația Angular rulată în browser, iar serverul este aplicația Node.js/Express care comunică cu baza de date MySQL. Comunicarea dintre frontend și backend se face prin cereri HTTP, iar datele sunt transmise în format JSON.

Această arhitectură este potrivită pentru proiect deoarece separă clar prezentarea de logica de business. Frontend-ul se ocupă de interacțiunea cu utilizatorul, validări de bază și afișarea datelor. Backend-ul se ocupă de autentificare, validare finală, reguli de programare, acces la baza de date și securitate.

| Strat           | Responsabilități                   | Exemple în VetCare                                     |
|-----------------|------------------------------------|--------------------------------------------------------|
| Prezentare      | Afișare UI, formulare, navigare    | Componente Angular: Home, Login, Programare, Dashboard |
| Servicii client | Comunicare HTTP și stare locală    | AuthService, HttpClient, token JWT                     |
| API backend     | Rute, validări, reguli de business | Express routes, middleware auth/admin                  |
| Persistență     | Stocare și interogare date         | MySQL: users, appointments, contact_messages           |

### 4.1 Arhitectura generală

Figura 1 poate fi reprezentată conceptual printr-un flux în patru niveluri: utilizatorul interacționează cu browserul, browserul rulează aplicația Angular, Angular trimite cereri către API-ul Express, iar API-ul citește sau modifică datele în MySQL. Pentru operațiile protejate, cererea conține headerul Authorization cu tokenul JWT.

În cazul autentificării, utilizatorul trimite email și parolă către backend. Serverul caută utilizatorul după email, compară parola cu hash-ul bcrypt și, dacă datele sunt corecte, emite un token semnat. Tokenul este păstrat pe client și transmis la cererile ulterioare. Middleware-ul verifică semnătura și extrage rolul.

În cazul programării, interfața afișează calendarul, iar pentru o dată selectată cere backend-ului lista orelor ocupate. La trimiterea programării, serverul verifică data, ziua săptămânii, ora, serviciul, tipul animalului și conflictul de slot. Doar după aceste validări se inserează în baza de date.

## 4.2 Organizarea frontend-ului

| Componentă    | Rol                                                         |
|---------------|-------------------------------------------------------------|
| home          | Pagina principală, prezentare servicii și motive de alegere |
| about         | Prezentarea cabinetului și echipei medicale                 |
| contact       | Formular public pentru mesaje                               |
| login         | Autentificare și înregistrare                               |
| programare    | Calendar și formular de programare                          |
| profile       | Date utilizator și istoric programări                       |
| dashboard     | Panou administrativ                                         |
| header/footer | Navigare și informații generale                             |

Structura componentelor este adecvată pentru o aplicație de dimensiune medie. Fiecare pagină are propriul fișier HTML pentru template, CSS pentru stilizare și TypeScript pentru logică. Această separare este utilă în mentenanță, deoarece modificarea designului nu necesită atingerea logicii de business.

Rutarea este definită în `app.routes.ts`. Ruta pentru programare este protejată cu `authGuard`, iar ruta pentru dashboard este protejată atât cu `authGuard`, cât și cu `adminGuard`. Astfel, accesul este controlat încă din interfață, iar backend-ul aplică verificarea finală.

## 4.3 Organizarea backend-ului

| Fișier/folder                    | Responsabilitate                                                    |
|----------------------------------|---------------------------------------------------------------------|
| <code>server.js</code>           | Configurare Express, middleware-uri globale, rute și pornire server |
| <code>routes</code>              | Definirea endpointurilor pe domenii funcționale                     |
| <code>controllers</code>         | Logică pentru autentificare și dashboard                            |
| <code>middleware</code>          | Verificare JWT și rol admin                                         |
| <code>config/db.js</code>        | Adaptor pentru interacțiunea cu MySQL                               |
| <code>utils/validation.js</code> | Funcții de validare și sanitizare                                   |

Organizarea backend-ului urmărește separarea regulilor de business de configurarea serverului. Rutele primesc cererile HTTP și apelează controllere sau logică specifică. Middleware-ul de autentificare este reutilizat pentru toate rutele protejate.

Un aspect important este faptul că `userId`-ul programării nu este primit din corpul cererii, ci este extras din token. Această decizie previne impersonarea, adică situația în care un utilizator ar încerca să creeze o programare în numele altui utilizator modificând manual corpul cererii.

## 5. Proiectarea bazei de date și a API-ului

Proiectarea bazei de date a urmărit păstrarea unui model simplu, ușor de explicat și suficient pentru fluxurile implementate. Entitățile principale sunt utilizatorul, programarea și mesajul de contact. Deși într-un sistem complet ar exista și entități pentru animale, fișe medicale, tratamente și facturi, acestea au fost lăsate ca direcții de extindere.

Cheile primare sunt de tip UUID stocat ca VARCHAR(36). Alegerea UUID-urilor oferă identificatori care nu sunt ușor de ghicit și care pot fi generați independent. Pentru dimensiunea aplicației, costul suplimentar de stocare nu reprezintă o problemă.

### 5.1 Schema relațională

| Tabel            | Câmpuri principale                                                                   | Rol               |
|------------------|--------------------------------------------------------------------------------------|-------------------|
| users            | id, email, password, name, role, createdAt                                           | Conturi și roluri |
| appointments     | id, userId, date, service, animalType, message, status, createdAt                    | Programări        |
| contact_messages | id, firstName, lastName, email, phone, animalType, subject, message, read, createdAt | Mesaje contact    |

Tabela users conține emailul unic, hash-ul parolei și rolul. Rolul poate fi user sau admin. Această informație este introdusă în tokenul JWT și este folosită ulterior pentru accesul la rutele administrative.

Tabela appointments include data și ora programării, serviciul, tipul animalului și statusul. Statusul poate fi pending, confirmed sau cancelled. Prin păstrarea statusului, aplicația nu este obligată să șteargă programările anulate, ceea ce permite păstrarea istoricului.

Tabela contact\_messages nu are cheie externă către users deoarece mesajele pot fi trimise și de vizitatori care nu au cont. Această decizie reflectă realitatea unui formular public, unde emailul introdus este doar o informație de contact și nu neapărat un cont existent.

### 5.2 Tipuri de chei folosite în scriptul MySQL

| Termen SQL  | Denumire      | Explicație în VetCare                                                                                                           |
|-------------|---------------|---------------------------------------------------------------------------------------------------------------------------------|
| Primary Key | Cheie primară | Identifică unic fiecare rând dintr-un tabel. În VetCare, câmpul id este PRIMARY KEY în users, appointments și contact_messages. |
| Foreign Key | Cheie externă | Leagă două tabele și impune integritatea                                                                                        |

|                   |                      |                                                                                                                                                                            |
|-------------------|----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                   |                      | referențială. appointments.userId referă users.id.                                                                                                                         |
| Unique Key        | Cheie unică          | Nu permite valori duplicate într-o coloană. În tabela users, email este UNIQUE pentru a preveni conturi duplicate.                                                         |
| Index / Key       | Index de căutare     | Accelerează interogările pe coloane folosite frecvent. În proiect apar indecși pe appointments.date, appointments.userId, contact_messages.email și contact_messages.read. |
| Surrogate Key     | Cheie surogat        | Cheie artificială, fără semnificație de business. UUID-ul id este o cheie surogat, deoarece nu descrie utilizatorul sau programarea.                                       |
| Natural Key       | Cheie naturală       | Câmp cu semnificație reală care poate identifica o entitate. Emailul poate fi privit ca o cheie naturală, dar în proiect nu este primary key, ci unique key.               |
| Composite Key     | Cheie compusă        | Cheie formată din mai multe coloane. Scriptul actual nu folosește chei compuse, deoarece fiecare tabel are id UUID separat.                                                |
| ON DELETE CASCADE | Acțiune referențială | Dacă un utilizator este șters, programările sale sunt șterse automat. Regula este definită pe cheia externă appointments.userId.                                           |

În scriptul MySQL, cuvântul KEY este folosit în mai multe contexte. PRIMARY KEY definește cheia principală a tabelului, UNIQUE KEY definește o constrângere de unicitate, iar KEY simplu definește un index folosit pentru performanță. De exemplu, KEY date\_idx (date) nu impune unicitate, ci ajută la căutarea rapidă a programărilor după dată.

Cheia externă appointments\_ibfk\_1 este importantă deoarece exprimă relația dintre utilizatori și programări. O programare trebuie să aparțină unui utilizator existent. Această regulă împiedică introducerea unor date orfane și păstrează coerența bazei de date.

Alegerea UUID-ului ca primary key este o decizie de proiectare. Alternativa ar fi fost un id numeric auto-increment. UUID-ul este mai greu de ghicit și este potrivit pentru aplicații care pot fi extinse ulterior, deși ocupă mai mult spațiu decât un număr întreg.

```
PRIMARY KEY (`id`)
UNIQUE KEY `email` (`email`)
KEY `userId` (`userId`)
KEY `date_idx` (`date`)
CONSTRAINT `appointments_ibfk_1`
```

FOREIGN KEY (`userId`) REFERENCES `users` (`id`) ON DELETE CASCADE

### 5.3 API REST

| Metodă | Endpoint                               | Acces      | Descriere                                    |
|--------|----------------------------------------|------------|----------------------------------------------|
| POST   | /api/auth/register                     | Public     | Creează un cont nou                          |
| POST   | /api/auth/login                        | Public     | Autentifică utilizatorul și returnează token |
| GET    | /api/auth/me                           | User       | Returnează utilizatorul curent               |
| PUT    | /api/auth/profile                      | User       | Actualizează numele                          |
| PUT    | /api/auth/change-password              | User       | Schimbă parola                               |
| GET    | /api/appointments/occupied             | User       | Returnează ore ocupate pentru o dată         |
| GET    | /api/appointments/mine                 | User       | Returnează programările proprii              |
| POST   | /api/appointments                      | User       | Creează programare                           |
| DELETE | /api/appointments/:id                  | User/Admin | Anulează programare                          |
| POST   | /api/contact                           | Public     | Trimite mesaj de contact                     |
| GET    | /api/dashboard/stats                   | Admin      | Statistici globale                           |
| GET    | /api/dashboard/users                   | Admin      | Lista utilizatorilor                         |
| PUT    | /api/dashboard/users/:id/role          | Admin      | Schimbă rol                                  |
| DELETE | /api/dashboard/users/:id               | Admin      | Șterge utilizator                            |
| GET    | /api/dashboard/appointments            | Admin      | Lista tuturor programărilor                  |
| PUT    | /api/dashboard/appointments/:id/status | Admin      | Schimbă status programare                    |
| GET    | /api/dashboard/messages                | Admin      | Lista mesajelor                              |

API-ul respectă un stil REST pragmatic. Resursele sunt grupate după domenii funcționale, iar metodele HTTP sunt folosite în mod intuitiv: POST pentru creare, GET pentru citire, PUT pentru actualizare și DELETE pentru anulare sau ștergere. Răspunsurile de eroare au un format simplu, cu proprietatea message.

Separarea endpointurilor publice de cele protejate este esențială. Formularul de contact și autentificarea sunt publice, dar programările și dashboard-ul necesită token. Dashboard-ul necesită și rol admin. Această structură previne accesarea accidentală sau intenționată a datelor administrative de către utilizatori obișnuiți.

## 5.4 Validări și reguli de business

Regulile de business sunt implementate pe server, chiar dacă unele sunt reflectate și în interfață. De exemplu, calendarul poate marca weekendurile ca indisponibile, dar backend-ul verifică din nou ziua săptămânii. Această dublă verificare este importantă deoarece datele trimise de client nu pot fi considerate de încredere.

La crearea unei programări, serverul verifică existența câmpurilor obligatorii, apartenența serviciului și a tipului de animal la listele acceptate, validitatea datei, faptul că data nu este în trecut, intervalul orar și conflictul de slot. Doar după aceste validări se creează înregistrarea.

Pentru autentificare, backend-ul normalizează emailul la litere mici și verifică formatul. Parola trebuie să aibă cel puțin opt caractere și să conțină literă și cifră. Aceste reguli nu înlocuiesc politici complexe de securitate, dar sunt suficiente pentru un proiect de licență și demonstrează atenție la validarea inputului.

## 6. Implementarea aplicației VetCare

Implementarea aplicației a fost realizată incremental, pornind de la schema bazei de date și de la fluxurile principale. Backend-ul a fost creat pentru a expune endpointurile necesare, iar frontend-ul a fost dezvoltat pentru a consuma aceste endpointuri prin HttpClient. În timpul implementării, s-a urmărit păstrarea mesajelor în limba română și coerența dintre validarea din interfață și validarea server-side.

Proiectul este împărțit în două directoare principale: `vetcare-backend` și `vetcare-frontend`. Această separare permite rularea independentă a serverului API și a aplicației Angular. Backend-ul rulează implicit pe portul 3000, iar frontend-ul pe portul 4200.

### 6.1 Implementarea autentificării

La înregistrare, parola este hash-uită cu `bcrypt` înainte de a fi stocată. În răspunsul către client, parola este eliminată din obiectul utilizatorului prin funcția `safeUser`. La login, parola primită este comparată cu hash-ul din baza de date, iar dacă verificarea reușește, serverul semnează un token JWT.

Tokenul conține id-ul utilizatorului, emailul și rolul. Aceste informații sunt suficiente pentru verificarea accesului la rutele protejate. Expirarea tokenului este configurabilă prin variabila `JWT_EXPIRES_IN`, cu valoare implicită de șapte zile.

```
function signToken(user) {  
  return jwt.sign(  
    { id: user.id, email: user.email, role: user.role },  
    process.env.JWT_SECRET,  
    { expiresIn: process.env.JWT_EXPIRES_IN || '7d' }  
  );  
}
```

### 6.2 Implementarea programărilor

Componenta Programare gestionează calendarul, data selectată, ora selectată și formularul de programare. Calendarul generează zilele lunii curente și marchează zilele trecute, weekendurile și ziua curentă. La selectarea unei zile valide, componenta cere backend-ului sloturile ocupate.

Lista orelor este predefinită din 30 în 30 de minute între 08:00 și 19:30. Butoanele pentru ore ocupate sunt dezactivate, ceea ce reduce riscul alegerii unui interval indisponibil. Totuși, serverul verifică din nou conflictul pentru a evita situația în care două cereri ajung aproape simultan.

La trimiterea formularului, componenta construiește un obiect `Date` care combină ziua selectată și ora selectată. Backend-ul primește data în format ISO, o validează și o transformă în format compatibil MySQL.

```
const conflict = all.find(a => {
  if (!a.date || a.status === 'cancelled') return false;
  return new Date(a.date).getTime() === apptDate.getTime();
});
if (conflict) {
  return res.status(409).json({ message: 'Slotul este deja ocupat.' });
}
```

### 6.3 Implementarea dashboard-ului administrativ

Dashboard-ul este disponibil doar administratorilor. Acesta oferă statistici globale, lista utilizatorilor, lista programărilor și lista mesajelor de contact. Prin intermediul dashboard-ului, administratorul poate modifica roluri, poate confirma sau anula programări și poate marca mesajele ca citite.

Backend-ul protejează toate rutele dashboard prin `requireAdmin`. Această funcție apelează mai întâi `requireAuth` și apoi verifică rolul din payload-ul tokenului. Dacă rolul nu este admin, răspunsul este 403.

Statisticile sunt utile pentru o vedere rapidă asupra activității cabinetului: numărul total de utilizatori, numărul de administratori, programări pending, confirmed sau cancelled și mesajele necitite. Aceste date pot fi extinse ulterior cu grafice, filtre pe intervale de timp sau rapoarte exportabile.

### 6.4 Interfața cu utilizatorul

Interfața aplicației urmărește un stil prietenos, potrivit unui cabinet veterinar. Paginile publice au rol de prezentare și includ informații despre servicii, echipă și contact. Paginile protejate au un caracter mai operațional, în special programarea și profilul.

Un principiu important al interfeței este reducerea ambiguității. Butoanele de acțiune folosesc texte clare, mesajele de eroare sunt în română, iar stările de încărcare sunt indicate prin textul „Se trimite...”. În calendar, zilele indisponibile sunt marcate vizual, iar ora selectată este evidențiată.

Imaginile aplicației sunt stocate în `src/assets/images` și sunt referite prin path-uri relative, astfel încât Angular să le copieze în build. Pentru versiunea curentă au fost adăugate imagini fără filigran, descărcate din surse gratuite, cu fișier separat pentru surse.

## **6.5 Securitate aplicată**

Securitatea aplicației este abordată pe mai multe niveluri. La nivel de autentificare, parolele nu sunt stocate în clar, iar tokenurile sunt semnate. La nivel de acces, middleware-ul verifică tokenul și rolul. La nivel de input, datele sunt validate și sanitizate. La nivel de trafic local, CORS este configurat pentru frontend-ul așteptat.

Rate limiting-ul este aplicat rutelor de login și register, cu limita de zece încercări în cincisprezece minute. Această măsură nu oprește toate atacurile, dar reduce riscul încercărilor brute-force simple și demonstrează o preocupare pentru protejarea endpointurilor sensibile.

Pentru prevenirea SQL injection, proiectul folosește `mysql2/promise` și interogări parametrizate în adaptorul de bază de date. De asemenea, `userId`-ul nu este acceptat din corpul cererii de programare, ci este extras din token, ceea ce reduce riscul accesului neautorizat la datele altui utilizator.

## 7. Testare, validare și evaluare

Testarea unei aplicații web full-stack trebuie să acopere atât interfața, cât și API-ul și baza de date. Pentru VetCare, testarea s-a concentrat pe fluxurile critice: înregistrare, autentificare, programare, anulare, acces dashboard și trimitere mesaj de contact. În plus, au fost verificate cazurile negative, cum ar fi parole slabe, date invalide sau acces neautorizat.

Validarea este importantă deoarece aplicația primește date de la utilizatori. Chiar dacă interfața limitează alegerile posibile, backend-ul trebuie să verifice independent fiecare cerere. Această abordare este obligatorie în aplicațiile web, unde clientul poate fi modificat sau ocolit.

| ID  | Scenariu                 | Date de intrare             | Rezultat așteptat          |
|-----|--------------------------|-----------------------------|----------------------------|
| T01 | Register cu date valide  | Email nou, parolă puternică | Cont creat, token returnat |
| T02 | Register email duplicat  | Email existent              | Răspuns 409                |
| T03 | Login valid              | Email și parolă corecte     | Token și user fără parolă  |
| T04 | Login invalid            | Parolă greșită              | Răspuns 401                |
| T05 | Programare zi lucrătoare | Data viitoare, slot liber   | Programare creată          |
| T06 | Programare weekend       | Sâmbătă sau duminică        | Răspuns 400                |
| T07 | Programare în trecut     | Data anterioară             | Răspuns 400                |
| T08 | Slot ocupat              | Aceeași dată și oră         | Răspuns 409                |
| T09 | Acces profil fără token  | Header lipsă                | Răspuns 401                |
| T10 | Acces dashboard ca user  | Token user                  | Răspuns 403                |
| T11 | Trimite mesaj contact    | Date publice valide         | Mesaj stocat               |
| T12 | Marcare mesaj citit      | Token admin                 | Status read actualizat     |

### 7.1 Testarea funcțională

Testarea funcțională urmărește dacă aplicația face ceea ce promite cerințele. În cazul VetCare, un flux complet începe cu înregistrarea unui utilizator, continuă cu autentificarea, selectarea unei programări și vizualizarea acesteia în profil. Administratorul poate apoi vedea programarea în dashboard și îi poate schimba statusul.

Un scenariu important este verificarea sloturilor ocupate. După crearea unei programări, endpointul `/api/appointments/occupied` trebuie să returneze ora respectivă pentru data aleasă. Interfața trebuie să dezactiveze butonul corespunzător, iar backend-ul trebuie să refuze o nouă programare pe același slot.

Testarea mesajelor de contact verifică faptul că formularul public funcționează fără autentificare, dar gestionarea mesajelor este disponibilă doar administratorului. Această separare confirmă că aplicația combină corect accesul public cu administrarea protejată.

## **7.2 Testarea securității**

Testarea securității a urmărit verificarea rutelor protejate și a validărilor. Cererile fără header Authorization către rutele /api/auth/me, /api/appointments/mine sau /api/dashboard/stats trebuie să primească răspuns 401. Cererile cu token de utilizator obișnuit către dashboard trebuie să primească 403.

Pentru autentificare, parolele invalide nu trebuie să permită login, iar mesajul returnat nu trebuie să indice dacă emailul sau parola a fost greșită. Această alegere reduce informațiile disponibile pentru un atacator.

Pentru parole, aplicația folosește bcrypt cu factor de lucru 10. Conform recomandărilor OWASP, bcrypt este o variantă acceptabilă pentru stocarea parolelor atunci când se folosește un factor de lucru suficient și parolele nu sunt stocate în clar.

## **7.3 Evaluarea soluției**

Soluția implementată îndeplinește obiectivele definite: oferă o interfață publică, autentificare, programări online, profil utilizator, formular de contact și dashboard administrativ. Din punct de vedere tehnic, aplicația demonstrează integrarea dintre frontend, backend și baza de date.

Punctele forte ale aplicației sunt simplitatea arhitecturii, claritatea fluxurilor și existența unor măsuri de securitate de bază. Punctele care pot fi îmbunătățite sunt lipsa unei entități separate pentru animale, lipsa notificărilor prin email, lipsa unui calendar administrativ avansat și lipsa testelor automate extinse.

Într-un context real, aplicația ar trebui completată cu politici de backup, logare structurată, HTTPS, protecție CSRF în funcție de metoda de stocare a tokenului, monitorizare și management al erorilor. Pentru scopul lucrării, însă, aplicația oferă un nucleu coerent și demonstrabil.

## 8. Concluzii și direcții viitoare

Lucrarea a prezentat procesul de proiectare și implementare a aplicației VetCare, o platformă web pentru gestionarea unui cabinet veterinar. Proiectul demonstrează utilizarea unei arhitecturi full-stack moderne, cu Angular pe client, Node.js/Express pe server și MySQL pentru persistența datelor.

Obiectivul principal a fost atins prin realizarea unui sistem funcțional care permite vizitatorilor să interacționeze cu cabinetul, utilizatorilor să se programeze online și administratorilor să gestioneze informațiile operaționale. Implementarea a evidențiat importanța separării responsabilităților, a validării server-side și a protejării rutelor sensibile.

Din punct de vedere academic, proiectul valorifică noțiuni de programare web, baze de date, inginerie software, securitate și testare. Contribuția proprie este vizibilă în definirea fluxurilor, implementarea programării online, proiectarea modelului de date și documentarea soluției.

Ca direcții viitoare, aplicația poate fi extinsă cu evidența animalelor, fișe medicale, încărcare documente, notificări automate prin email sau SMS, calendar avansat pentru administratori, rapoarte statistice, export PDF și integrarea plăților online. O altă direcție importantă este adăugarea testelor automate end-to-end și a unui pipeline de deploy.

În concluzie, VetCare reprezintă o soluție educațională completă pentru un scenariu realist. Aplicația nu înlocuiește un sistem clinic profesional, dar oferă o bază solidă pentru digitalizarea operațiilor simple ale unui cabinet veterinar și pentru continuarea dezvoltării într-un produs mai complex.

## Bibliografie

- [1] Facultatea de Matematică și Informatică, Universitatea „Ovidius” din Constanța, Ghid de elaborare și susținere a lucrării de licență și disertație, 2025, <https://fmi.univ-ovidius.ro/wp-content/uploads/2025/05/Ghid-de-elaborare-a-lucrarii-de-finalizare-studii-FMI2025.pdf>
- [2] Facultatea de Matematică și Informatică, Organizarea Examenului de Licență, <https://fmi.univ-ovidius.ro/en/academic/studii-de-licenta/licenta-documente/>
- [3] Angular Documentation, Define routes, <https://angular.dev/guide/routing/define-routes>
- [4] Angular Documentation, Component API, <https://angular.dev/api/core/Component>
- [5] Express.js Documentation, Routing, <https://expressjs.com/en/guide/routing.html>
- [6] Node.js Documentation, Introduction to Node.js, <https://nodejs.org/en/learn/getting-started/introduction-to-nodejs>
- [7] MySQL 8.4 Reference Manual, FOREIGN KEY Constraints, <https://dev.mysql.com/doc/refman/8.4/en/create-table-foreign-keys.html>
- [8] RFC Editor, RFC 7519: JSON Web Token (JWT), <https://www.rfc-editor.org/rfc/rfc7519>
- [9] OWASP Cheat Sheet Series, Password Storage Cheat Sheet, [https://cheatsheetseries.owasp.org/cheatsheets/Password\\_Storage\\_Cheat\\_Sheet.html](https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html)
- [10] MDN Web Docs, Cross-Origin Resource Sharing (CORS), <https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS>
- [11] VetCare, README.md și documentația tehnică locală a proiectului: API\_ENDPOINTS.md, ER\_DIAGRAM.md, vetcare\_schema.sql.

## Anexa A. Instrucțiuni de instalare și rulare

Pentru rularea aplicației sunt necesare Node.js, npm și MySQL. Baza de date se creează folosind scriptul `vetcare_schema.sql`. Backend-ul se configurează prin fișierul `.env`, unde se setează portul, secretul JWT, datele de conectare la MySQL și URL-ul frontend-ului.

Pașii generali sunt: crearea bazei de date, instalarea dependențelor backend, popularea opțională cu date demo, pornirea backend-ului, instalarea dependențelor frontend și pornirea aplicației Angular. Pentru testare rapidă, proiectul include conturi demo generate de `seed.js`.

```
mysql -u root -p < vetcare_schema.sql
cd vetcare-backend
npm install
npm run seed
npm start

cd ../vetcare-frontend
npm install
npm start
```

## Anexa B. Scenarii de test detaliate

| ID   | Zonă          | Acțiune                                    | Rezultat așteptat                                          |
|------|---------------|--------------------------------------------|------------------------------------------------------------|
| TB01 | Programări    | verificarea validării pentru câmpuri lipsă | răspuns 400 cu mesaj în limba română                       |
| TB02 | Dashboard     | verificarea accesului fără token           | răspuns 401 și blocarea operației                          |
| TB03 | Contact       | verificarea accesului cu rol insuficient   | răspuns 403 pentru resurse administrative                  |
| TB04 | Profil        | verificarea persistării datelor în MySQL   | înregistrarea este creată, actualizată sau filtrată corect |
| TB05 | Autentificare | verificarea răspunsului pentru date valide | răspuns 200 sau 201 și payload JSON corect                 |
| TB06 | Programări    | verificarea validării pentru câmpuri lipsă | răspuns 400 cu mesaj în limba română                       |
| TB07 | Dashboard     | verificarea accesului fără token           | răspuns 401 și blocarea operației                          |
| TB08 | Contact       | verificarea accesului cu rol insuficient   | răspuns 403 pentru resurse administrative                  |
| TB09 | Profil        | verificarea persistării datelor în MySQL   | înregistrarea este creată, actualizată sau filtrată corect |
| TB10 | Autentificare | verificarea răspunsului pentru             | răspuns 200 sau 201 și payload                             |

|      |               |                                            |                                                            |
|------|---------------|--------------------------------------------|------------------------------------------------------------|
|      |               | date valide                                | JSON corect                                                |
| TB11 | Programări    | verificarea validării pentru câmpuri lipsă | răspuns 400 cu mesaj în limba română                       |
| TB12 | Dashboard     | verificarea accesului fără token           | răspuns 401 și blocarea operației                          |
| TB13 | Contact       | verificarea accesului cu rol insuficient   | răspuns 403 pentru resurse administrative                  |
| TB14 | Profil        | verificarea persistării datelor în MySQL   | înregistrarea este creată, actualizată sau filtrată corect |
| TB15 | Autentificare | verificarea răspunsului pentru date valide | răspuns 200 sau 201 și payload JSON corect                 |
| TB16 | Programări    | verificarea validării pentru câmpuri lipsă | răspuns 400 cu mesaj în limba română                       |
| TB17 | Dashboard     | verificarea accesului fără token           | răspuns 401 și blocarea operației                          |
| TB18 | Contact       | verificarea accesului cu rol insuficient   | răspuns 403 pentru resurse administrative                  |
| TB19 | Profil        | verificarea persistării datelor în MySQL   | înregistrarea este creată, actualizată sau filtrată corect |
| TB20 | Autentificare | verificarea răspunsului pentru date valide | răspuns 200 sau 201 și payload JSON corect                 |
| TB21 | Programări    | verificarea validării pentru câmpuri lipsă | răspuns 400 cu mesaj în limba română                       |
| TB22 | Dashboard     | verificarea accesului fără token           | răspuns 401 și blocarea operației                          |
| TB23 | Contact       | verificarea accesului cu rol insuficient   | răspuns 403 pentru resurse administrative                  |
| TB24 | Profil        | verificarea persistării datelor în MySQL   | înregistrarea este creată, actualizată sau filtrată corect |
| TB25 | Autentificare | verificarea răspunsului pentru date valide | răspuns 200 sau 201 și payload JSON corect                 |
| TB26 | Programări    | verificarea validării pentru câmpuri lipsă | răspuns 400 cu mesaj în limba română                       |
| TB27 | Dashboard     | verificarea accesului fără token           | răspuns 401 și blocarea operației                          |
| TB28 | Contact       | verificarea accesului cu rol insuficient   | răspuns 403 pentru resurse administrative                  |
| TB29 | Profil        | verificarea persistării datelor în MySQL   | înregistrarea este creată, actualizată sau filtrată corect |
| TB30 | Autentificare | verificarea răspunsului pentru date valide | răspuns 200 sau 201 și payload JSON corect                 |

## Anexa C. Fragment de schemă SQL

```
CREATE TABLE `appointments` (  
  `id`          VARCHAR(36) NOT NULL,  
  `userId`      VARCHAR(36)          DEFAULT NULL,  
  `date`        DATETIME             DEFAULT NULL,  
  `service`     VARCHAR(255)         DEFAULT NULL,  
  `animalType` VARCHAR(255)         DEFAULT '',  
  `message`     VARCHAR(1000)        DEFAULT '',  
  `status`      ENUM('pending','confirmed','cancelled') DEFAULT 'pending',  
  `createdAt`   DATETIME             DEFAULT CURRENT_TIMESTAMP,  
  PRIMARY KEY (`id`),  
  KEY `userId` (`userId`),  
  KEY `date_idx` (`date`),  
  CONSTRAINT `appointments_ibfk_1`  
    FOREIGN KEY (`userId`) REFERENCES `users` (`id`) ON DELETE CASCADE  
);
```

## Anexa D. Întrebări posibile pentru susținere

- Care este rolul JWT în aplicație și ce informații sunt incluse în token?
- De ce validările importante sunt realizate pe backend, chiar dacă există validări în frontend?
- Cum este prevenită programarea pe un slot deja ocupat?
- Care este diferența dintre rolul user și rolul admin în VetCare?
- De ce parolele nu sunt stocate în clar în baza de date?
- Ce avantaje oferă separarea frontend/backend?
- Cum poate fi extinsă aplicația pentru fișe medicale ale animalelor?
- Ce rol au cheile externe în schema MySQL?
- Ce limitări are aplicația în forma curentă?
- Ce teste automate ar fi utile pentru o versiune de producție?

Aceste întrebări sunt orientative și urmăresc pregătirea pentru discuția cu comisia. Răspunsurile trebuie susținute prin demonstrația aplicației și prin referire la capitolele tehnice ale lucrării.

## Anexa F. Capturi de cod și scripturi relevante

În această anexă sunt incluse capturi generate din fișierele sursă ale proiectului. Ele arată concret unde apar HTML, CSS, TypeScript, controllere, modele, rute URL și scriptul SQL al bazei de date.

```
Script SQL - crearea bazei de date VetCare

01  -- -----
02  -- VetCare - Schema MySQL
03  -- Ruleaza: mysql -u root -p < vetcare_schema.sql
04  -- -----
05
06  CREATE DATABASE IF NOT EXISTS `vetcare`
07  /*!40100 DEFAULT CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci */;
08  USE `vetcare`;
09
10  -- -- Users --
11  DROP TABLE IF EXISTS `appointments`;
12  DROP TABLE IF EXISTS `contact_messages`;
13  DROP TABLE IF EXISTS `users`;
14
15  CREATE TABLE `users` (
16    `id` VARCHAR(36) NOT NULL,
17    `email` VARCHAR(255) NOT NULL,
18    `password` VARCHAR(255) NOT NULL,
19    `name` VARCHAR(255) DEFAULT '',
20    `role` ENUM('user','admin') DEFAULT 'user',
21    `createdAt` DATETIME DEFAULT CURRENT_TIMESTAMP,
22    PRIMARY KEY (`id`),
23    UNIQUE KEY `email` (`email`)
24  ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
25
26  -- -- Appointments --
27  CREATE TABLE `appointments` (
28    `id` VARCHAR(36) NOT NULL,
29    `userId` VARCHAR(36) DEFAULT NULL,
30    `date` DATETIME DEFAULT NULL,
31    `service` VARCHAR(255) DEFAULT NULL,
32    `animalType` VARCHAR(255) DEFAULT '',
33    `message` VARCHAR(1000) DEFAULT '',
34    `status` ENUM('pending','confirmed','cancelled') DEFAULT 'pending',
35    `createdAt` DATETIME DEFAULT CURRENT_TIMESTAMP,
36    PRIMARY KEY (`id`),
37    KEY `userId` (`userId`),
38    KEY `date_idx` (`date`),
39    CONSTRAINT `appointments_ibfk_1`
40    FOREIGN KEY (`userId`) REFERENCES `users` (`id`) ON DELETE CASCADE
41  ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
42
```

Figura F.1. Script SQL pentru baza de date

## TypeScript - componenta Programare

```
01 import { Component, OnInit } from '@angular/core';
02 import { CommonModule } from '@angular/common';
03 import { FormsModule } from '@angular/forms';
04 import { HttpClient, HttpHeaders } from '@angular/common/http';
05 import { AuthService } from '../services/auth.service';
06 import { Router } from '@angular/router';
07
08 @Component({
09   selector: 'app-programare',
10   standalone: true,
11   imports: [CommonModule, FormsModule],
12   templateUrl: './programare.component.html',
13   styleUrls: ['./programare.component.css']
14 })
15 export class ProgramareComponent implements OnInit {
16   private apiUrl = 'http://localhost:3000/api';
17
18   // Calendar
19   currentDate = new Date();
20   selectedDate: Date | null = null;
21   selectedHour: string | null = null;
22   calendarDays: any[] = [];
23   occupiedSlots: string[] = [];
24
25   // Form
26   form = {
27     animalType: '',
28     service: '',
29     message: ''
30   };
31
32   animalTypes = ['Căine', 'Pisică', 'Iepure', 'Pasăre', 'Reptilă', 'Animal Exotic', 'Altul'];
33   services = ['Consultație Generală', 'Vaccinare', 'Chirurgie', 'Radiologie', 'Laborator', 'Grooming', 'Altul'];
34   hours = ['08:00', '08:30', '09:00', '09:30', '10:00', '10:30', '11:00', '11:30',
35     '12:00', '12:30', '13:00', '13:30', '14:00', '14:30', '15:00', '15:30',
36     '16:00', '16:30', '17:00', '17:30', '18:00', '18:30', '19:00', '19:30'];
37
38   submitted = false;
39   loading = false;
40   error = '';
41
42   constructor(private http: HttpClient, private auth: AuthService, private router: Router) {}
```

Figura F.2. Logică TypeScript pentru programări

## HTML - template Programare

```

01 <div class="page-hero">
02   <div class="container">
03     <span class="section-label">Rezervă o vizită</span>
04     <h1>Programare Online</h1>
05     <p>Alege data și ora care îți convine</p>
06   </div>
07 </div>
08
09 <section class="section">
10   <div class="container">
11
12     <!-- Success -->
13     <div class="success-card" *ngIf="submitted">
14       <i class="fa-solid fa-circle-check"></i>
15       <h2>Programare confirmată!</h2>
16       <p>Te vom contacta pentru confirmare. Ne vedem curând! []</p>
17       <a routerLink="/" class="btn btn-primary">Înapoi Acasă</a>
18     </div>
19
20     <div class="programare-layout" *ngIf="!submitted">
21
22       <!-- Calendar -->
23       <div class="calendar-section">
24         <div class="calendar-card">
25           <div class="cal-header">
26             <button class="cal-nav" (click)="prevMonth()"><i class="fa-solid fa-chevron-left"></i></button>
27             <h3>{{ getMonthName() }}</h3>
28             <button class="cal-nav" (click)="nextMonth()"><i class="fa-solid fa-chevron-right"></i></button>
29           </div>
30
31           <div class="cal-weekdays">
32             <span>Lun</span><span>Mar</span><span>Mie</span>
33             <span>Joi</span><span>Vin</span>
34             <span class="weekend">Sâm</span><span class="weekend">Dum</span>
35           </div>
36
37           <div class="cal-grid">
38             <div *ngFor="let day of calendarDays"
39               class="cal-day"
40               [class.empty]="!day"
41               [class.past]="day?.isPast"
42               [class.weekend]="day?.isWeekend">

```

Figura F.3. Template HTML pentru calendarul de programări

## CSS - stilizare Programare

```
01 .programare-layout {
02   display: grid;
03   grid-template-columns: 1.2fr 1fr;
04   gap: 40px;
05   align-items: start;
06 }
07
08 /* Calendar */
09 .calendar-card {
10   background: #fff;
11   border-radius: 16px;
12   padding: 28px;
13   box-shadow: 0 4px 24px rgba(45,80,22,0.12);
14   margin-bottom: 24px;
15 }
16
17 .cal-header {
18   display: flex;
19   align-items: center;
20   justify-content: space-between;
21   margin-bottom: 24px;
22 }
23 .cal-header h3 {
24   font-family: 'Playfair Display', serif;
25   font-size: 1.1rem;
26   color: #2D5016;
27   text-transform: capitalize;
28 }
29 .cal-nav {
30   width: 36px; height: 36px;
31   border-radius: 50%;
32   border: 2px solid #F0EBE1;
33   background: none;
34   cursor: pointer;
35   color: #2D5016;
36   display: flex; align-items: center; justify-content: center;
37   transition: all 0.3s;
38 }
39 .cal-nav:hover { background: #2D5016; color: #fff; border-color: #2D5016; }
40
41 .cal-weekdays {
42   display: grid;
```

*Figura F.4. Stilizare CSS pentru pagina Programare*

#### Controller - autentificare

```
01 const bcrypt = require('bcryptjs');
02 const jwt = require('jsonwebtoken');
03 const db = require('../config/db');
04 const { createUser } = require('../models/user.model');
05 const { isValidEmail, isStrongPassword, sanitizeText } = require('../utils/validation');
06
07 function signToken(user) {
08   return jwt.sign(
09     { id: user.id, email: user.email, role: user.role },
10     process.env.JWT_SECRET,
11     { expiresIn: process.env.JWT_EXPIRES_IN || '7d' }
12   );
13 }
14
15 function safeUser(user) {
16   if (!user) return null;
17   const { password, ...rest } = user;
18   return rest;
19 }
20
21 async function register(req, res) {
22   try {
23     const email = sanitizeText(req.body.email, 254).toLowerCase();
24     const name = sanitizeText(req.body.name, 100);
25     const password = req.body.password;
26
27     if (!email || !password) {
28       return res.status(400).json({ message: 'Email și parola sunt obligatorii.' });
29     }
30     if (!isValidEmail(email)) {
31       return res.status(400).json({ message: 'Email invalid.' });
32     }
33     if (!isStrongPassword(password)) {
34       return res.status(400).json({
35         message: 'Parola trebuie să aibă minim 8 caractere, cu cel puțin o literă și o cifră.'
36       });
37     }
38
39     const exists = await db.findOne('users', { email });
40     if (exists) return res.status(409).json({ message: 'Există deja un cont cu acest email.' });
41
42     const hash = await bcrypt.hash(password, 10);
```

Figura F.5. Controller backend pentru register/login

#### Model - utilizator

```
01 const { randomUUID } = require('crypto');
02
03 function createUser({ email, password, name = '', role = 'user' }) {
04   return {
05     id: randomUUID(),
06     email: email.toLowerCase().trim(),
07     password,
08     name: name.trim(),
09     role,
10     createdAt: new Date().toISOString().slice(0, 19).replace('T', ' ')
11   };
12 }
13
14 module.exports = { createUser };
```

Figura F.6. Model backend pentru utilizator

#### URL/Router - rute Angular

```
01 import { Routes } from '@angular/router';
02 import { HomeComponent } from './components/home/home.component';
03 import { AboutComponent } from './components/about/about.component';
04 import { ContactComponent } from './components/contact/contact.component';
05 import { LoginComponent } from './components/login/login.component';
06 import { DashboardComponent } from './components/dashboard/dashboard.component';
07 import { ProfileComponent } from './components/profile/profile.component';
08 import { authGuard } from './guards/auth.guard';
09 import { adminGuard } from './guards/admin.guard';
10 import { ProgramareComponent } from './components/programare/programare.component';
11 export const routes: Routes = [
12   { path: 'programare', component: ProgramareComponent, canActivate: [authGuard] },
13   { path: '', component: HomeComponent },
14   { path: 'about', component: AboutComponent },
15   { path: 'contact', component: ContactComponent },
16   { path: 'login', component: LoginComponent },
17   { path: 'dashboard', component: DashboardComponent, canActivate: [authGuard, adminGuard] },
18   { path: 'profile', component: ProfileComponent, canActivate: [authGuard] },
19   { path: '**', redirectTo: '' }
20 ];
```

Figura F.7. Definirea rutelor frontend

#### URL/API - rute programări

```
01 const express = require('express');
02 const router = express.Router();
03 const crypto = require('crypto');
04 const db = require('./config/db');
05 const { requireAuth } = require('./middleware/auth.middleware');
06 const { sanitizeText } = require('./utils/validation');
07
08 const VALID_SERVICES = [
09   'Consultație Generală', 'Vaccinare', 'Chirurgie',
10   'Radiologie', 'Laborator', 'Grooming', 'Altul'
11 ];
12 const VALID_ANIMALS = [
13   'Câine', 'Pisică', 'Iepure', 'Pasăre',
14   'Reptilă', 'Animal Exotic', 'Altul'
15 ];
16
17 // GET /api/appointments/occupied?date=YYYY-MM-DD
18 router.get('/occupied', requireAuth, async (req, res) => {
19   try {
20     const date = req.query.date;
21     if (!date) return res.json([]);
22
23     const all = await db.findAll('appointments');
24     const occupied = all
25       .filter(a => {
26         if (!a.date || a.status === 'cancelled') return false;
27         const d = new Date(a.date);
28         const dStr = d.toISOString().slice(0, 10);
29         return dStr === date;
30       })
31       .map(a => {
32         const d = new Date(a.date);
33         return `${String(d.getHours()).padStart(2, '0')}:${String(d.getMinutes()).padStart(2, '0')}`;
34       });
35
36     return res.json(occupied);
37   } catch (err) {
38     console.error('[occupied]', err);
39     return res.status(500).json({ message: 'Eroare server.' });
40   }
41 });
42
```

Figura F.8. Endpointuri backend pentru programări

#### Angular HttpClient - serviciu autentificare

```
01 import { Injectable } from '@angular/core';
02 import { HttpClient } from '@angular/common/http';
03 import { Router } from '@angular/router';
04 import { tap } from 'rxjs/operators';
05
06 @Injectable({ providedIn: 'root' })
07 export class AuthService {
08
09   private apiUrl = 'http://localhost:3000/api';
10
11   constructor(private http: HttpClient, private router: Router) {}
12
13   login(email: string, password: string) {
14     return this.http.post<any>(`${this.apiUrl}/auth/login`, { email, password }).pipe(
15       tap(res => {
16         localStorage.setItem('token', res.token);
17         localStorage.setItem('user', JSON.stringify(res.user));
18       })
19     );
20   }
21
22   register(email: string, password: string, name: string) {
23     return this.http.post<any>(`${this.apiUrl}/auth/register`, { email, password, name }).pipe(
24       tap(res => {
25         localStorage.setItem('token', res.token);
26         localStorage.setItem('user', JSON.stringify(res.user));
27       })
28     );
29   }
30
31   logout() {
32     localStorage.removeItem('token');
33     localStorage.removeItem('user');
34     this.router.navigate(['/login']);
35   }
36
37   getToken() { return localStorage.getItem('token'); }
38   getUser() { return JSON.parse(localStorage.getItem('user') || 'null'); }
39   isLoggedIn() { return !!this.getToken(); }
40   isAdmin() { return this.getUser()?.role === 'admin'; }
41 }
```

Figura F.9. Folosirea HttpClient pentru apeluri API

#### Middleware - verificare JWT

```
01 /**
02  * Auth Middleware - verifica JWT pe rutele protejate
03  */
04 const jwt = require('jsonwebtoken');
05
06 function requireAuth(req, res, next) {
07   const header = req.headers.authorization;
08   if (!header || !header.startsWith('Bearer ')) {
09     return res.status(401).json({ message: 'Token lipsa. Autentifica-te.' });
10   }
11
12   const token = header.split(' ')[1];
13   try {
14     const payload = jwt.verify(token, process.env.JWT_SECRET);
15     req.user = payload; // { id, email, role }
16     next();
17   } catch {
18     return res.status(401).json({ message: 'Token invalid sau expirat.' });
19   }
20 }
21
22 function requireAdmin(req, res, next) {
23   requireAuth(req, res, () => {
24     if (req.user.role !== 'admin') {
25       return res.status(403).json({ message: 'Acces interzis. Doar admini.' });
26     }
27     next();
28   });
29 }
30
31 module.exports = { requireAuth, requireAdmin };
```

Figura F.10. Middleware Express pentru autentificare și rol admin

## Server API - Express, CORS, rate limit

```
01  /**
02  * VetCare Backend - server.js
03  * Node.js + Express + MySQL
04  */
05  require('dotenv').config();
06  const express = require('express');
07  const cors = require('cors');
08  const rateLimit = require('express-rate-limit');
09
10  const authRoutes = require('./routes/auth.routes');
11  const dashboardRoutes = require('./routes/dashboard.routes');
12  const appointmentsRoutes = require('./routes/appointments.routes');
13  const contactRoutes = require('./routes/contact.routes');
14
15  const app = express();
16  const PORT = process.env.PORT || 3000;
17
18  /* — Middleware global — */
19  app.use(cors({
20    origin: process.env.FRONTEND_URL || 'http://localhost:4200',
21    credentials: true
22  }));
23  app.use(express.json());
24
25  /* — Rate limiting —
26  * Apara endpoint-urile sensibile (login/register) impotriva atacurilor brute-force.
27  */
28  const authLimiter = rateLimit({
29    windowMs: 15 * 60 * 1000, // 15 minute
30    max: 10, // max 10 incercari / IP / fereastra
31    message: { message: 'Prea multe incercări. Încearcă din nou în 15 minute.' },
32    standardHeaders: true,
33    legacyHeaders: false
34  });
35
36  const generalLimiter = rateLimit({
37    windowMs: 15 * 60 * 1000,
38    max: 200,
39    standardHeaders: true,
40    legacyHeaders: false
41  });
42
```

Figura F.11. Configurarea API-ului Express

## Adaptor bază de date - MySQL

```
01  /**
02  * VetCare DB Adapter - MySQL
03  *
04  * Expune metode generice (findOne / findAll / insert / update / delete)
05  * peste pool-ul mysql2, ca sa pastram codul controllerelor curat.
06  */
07
08  require('dotenv').config();
09
10  class MySQLAdapter {
11    constructor() {
12      const mysql = require('mysql2/promise');
13      this.pool = mysql.createPool({
14        host: process.env.DB_HOST || 'localhost',
15        port: process.env.DB_PORT || 3306,
16        database: process.env.DB_NAME || 'vetcare',
17        user: process.env.DB_USER || 'root',
18        password: process.env.DB_PASS || '',
19        waitForConnections: true,
20        connectionLimit: 10
21      });
22      console.log(`[DB] MySQL conectat -> ${process.env.DB_NAME || 'vetcare'}`);
23    }
24
25    async findOne(collection, predicate) {
26      const entries = Object.entries(predicate);
27      const where = entries.map(([k]) => `${k}\` = ?`).join(' AND ');
28      const values = entries.map([l, v] => v);
29      const [rows] = await this.pool.query(
30        `SELECT * FROM \`${collection}\` WHERE ${where} LIMIT 1`,
31        values
32      );
33      return rows[0] || null;
34    }
35
36    async findAll(collection, predicate = {}) {
37      const entries = Object.entries(predicate);
38      if (!entries.length) {
39        const [rows] = await this.pool.query(`SELECT * FROM \`${collection}\``);
40        return rows;
41      }
42      const where = entries.map(([k]) => `${k}\` = ?`).join(' AND ');
```

Figura F.12. Funcții de acces la baza de date

## **Anexa E. Analiză extinsă a deciziilor de proiectare**

### **Separarea responsabilităților**

Separarea responsabilităților reduce complexitatea locală a fiecărei componente. În VetCare, interfața Angular nu construiește interogări SQL și nu decide drepturi administrative, iar backend-ul nu conține markup-ul paginilor. Această separare permite înlocuirea unei părți fără rescrierea completă a celeilalte.

Separarea responsabilităților reduce complexitatea locală a fiecărei componente. În aplicația, interfața Angular nu construiește interogări SQL și nu decide drepturi administrative, iar backend-ul nu conține markup-ul paginilor. Această separare permite înlocuirea unei părți fără rescrierea completă a celeilalte.

Decizia prezentată are efect direct asupra modului în care aplicația poate fi înțeleasă, testată și extinsă. Într-o lucrare de licență, nu este suficient ca produsul să funcționeze; este necesar ca deciziile să fie motivate, iar cititorul să poată urmări legătura dintre cerințe, proiectare, implementare și testare.

### **Validarea datelor**

Validarea datelor este una dintre cele mai importante activități într-un sistem care primește input de la utilizatori. Proiectul tratează emailul, parola, data programării, serviciul și tipul animalului ca date ce trebuie verificate explicit. Această abordare reduce riscul stocării unor valori invalide.

Validarea datelor este una dintre cele mai importante activități într-un sistem care primește input de la utilizatori. Proiectul tratează emailul, parola, data programării, serviciul și tipul animalului ca date ce trebuie verificate explicit. Această abordare reduce riscul stocării unor valori invalide.

Decizia prezentată are efect direct asupra modului în care aplicația poate fi înțeleasă, testată și extinsă. Într-o lucrare de licență, nu este suficient ca produsul să funcționeze; este necesar ca deciziile să fie motivate, iar cititorul să poată urmări legătura dintre cerințe, proiectare, implementare și testare.

## **Modelul de roluri**

Modelul de roluri este simplu, dar suficient pentru scenariul ales. Rolul user permite folosirea aplicației de către clienți, iar rolul admin permite gestionarea operațiilor cabinetului. Pentru o versiune viitoare s-ar putea introduce roluri suplimentare, cum ar fi medic, recepționar sau manager.

Modelul de roluri este simplu, dar suficient pentru scenariul ales. Rolul user permite folosirea aplicației de către clienți, iar rolul admin permite gestionarea operațiilor cabinetului. Pentru o versiune viitoare s-ar putea introduce roluri suplimentare, cum ar fi medic, recepționar sau manager.

Decizia prezentată are efect direct asupra modului în care aplicația poate fi înțeleasă, testată și extinsă. Într-o lucrare de licență, nu este suficient ca produsul să funcționeze; este necesar ca deciziile să fie motivate, iar cititorul să poată urmări legătura dintre cerințe, proiectare, implementare și testare.

## **Persistența datelor**

Persistența în MySQL face ca programările și mesajele să fie păstrate între reporniri. Alegerea unui model relațional este potrivită deoarece datele au structură clară și relații stabile. Indecșii pe email, dată și read ajută la interogări frecvente.

Persistența în MySQL face ca programările și mesajele să fie păstrate între reporniri. Alegerea unui model relațional este potrivită deoarece datele au structură clară și relații stabile. Indecșii pe email, dată și read ajută la interogări frecvente.

Decizia prezentată are efect direct asupra modului în care aplicația poate fi înțeleasă, testată și extinsă. Într-o lucrare de licență, nu este suficient ca produsul să funcționeze; este necesar ca deciziile să fie motivate, iar cititorul să poată urmări legătura dintre cerințe, proiectare, implementare și testare.

## **Experiența utilizatorului**

Experiența utilizatorului este influențată de claritatea formularelor și de feedback. În VetCare, textele sunt în limba română, iar acțiunile au mesaje explicite. Calendarul de programări reduce numărul erorilor prin marcarea zilelor indisponibile și prin dezactivarea sloturilor ocupate.

Experiența utilizatorului este influențată de claritatea formularelor și de feedback. În aplicația, textele sunt în limba română, iar acțiunile au mesaje explicite. Calendarul de programări reduce numărul erorilor prin marcarea zilelor indisponibile și prin dezactivarea sloturilor ocupate.

Decizia prezentată are efect direct asupra modului în care aplicația poate fi înțeleasă, testată și extinsă. Într-o lucrare de licență, nu este suficient ca produsul să funcționeze; este necesar ca deciziile să fie motivate, iar cititorul să poată urmări legătura dintre cerințe, proiectare, implementare și testare.

## **Extensibilitate**

Extensibilitatea este posibilă prin adăugarea de entități noi și endpointuri noi. De exemplu, tabela `animals` ar putea lega fiecare utilizator de unul sau mai multe animale, iar tabela `medical_records` ar putea păstra consultații și tratamente. Structura actuală permite aceste extensii fără schimbări radicale.

Extensibilitatea este posibilă prin adăugarea de entități noi și endpointuri noi. De exemplu, tabela `animals` ar putea lega fiecare utilizator de unul sau mai multe animale, iar tabela `medical_records` ar putea păstra consultații și tratamente. Structura actuală permite aceste extensii fără schimbări radicale.

Decizia prezentată are efect direct asupra modului în care aplicația poate fi înțeleasă, testată și extinsă. Într-o lucrare de licență, nu este suficient ca produsul să funcționeze; este necesar ca deciziile să fie motivate, iar cititorul să poată urmări legătura dintre cerințe, proiectare, implementare și testare.

## **Limitări**

Limitările proiectului sunt normale pentru o lucrare de licență. Nu există încă notificări automate, nu există un sistem complet de fișe medicale, iar autentificarea nu folosește refresh tokens. Aceste limitări sunt documentate pentru a delimita clar contribuția curentă de direcțiile viitoare.

Limitările sistemului sunt normale pentru o lucrare de licență. Nu există încă notificări automate, nu există un sistem complet de fișe medicale, iar autentificarea nu folosește refresh tokens. Aceste limitări sunt documentate pentru a delimita clar contribuția curentă de direcțiile viitoare.

Decizia prezentată are efect direct asupra modului în care aplicația poate fi înțeleasă, testată și extinsă. Într-o lucrare de licență, nu este suficient ca produsul să funcționeze; este necesar ca deciziile să fie motivate, iar cititorul să poată urmări legătura dintre cerințe, proiectare, implementare și testare.

## **Calitatea codului**

Calitatea codului este susținută prin structurarea fișierelor și prin folosirea unor biblioteci cunoscute. Totuși, o variantă de producție ar trebui să includă testare automată mai amplă, linting strict, logare structurată și un sistem de migrații pentru baza de date.

Calitatea codului este susținută prin structurarea fișierelor și prin folosirea unor biblioteci cunoscute. Totuși, o variantă de producție ar trebui să includă testare automată mai amplă, linting strict, logare structurată și un sistem de migrații pentru baza de date.

Decizia prezentată are efect direct asupra modului în care aplicația poate fi înțeleasă, testată și extinsă. Într-o lucrare de licență, nu este suficient ca produsul să funcționeze; este necesar ca deciziile să fie motivate, iar cititorul să poată urmări legătura dintre cerințe, proiectare, implementare și testare.